

Lineare Modelle für Regression

Contents

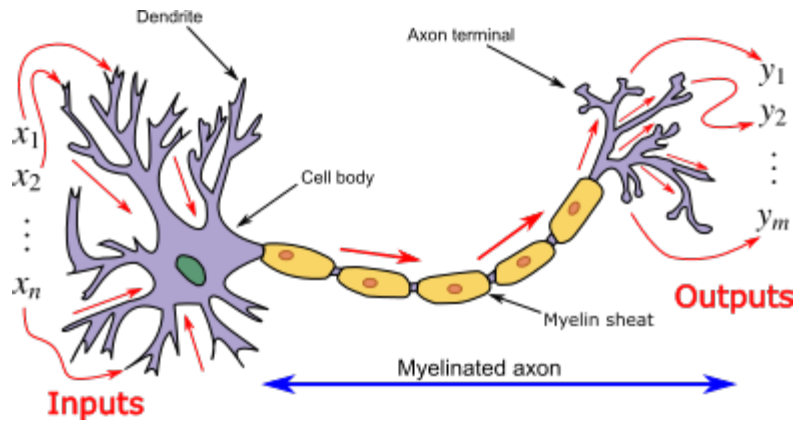
- Neuronale Netze (NN)
- Lineare Neuronale Netze für Regression
- Lineare Regression
- Aufgaben

Lernziele

- Exkurs: Lineare Neuronale Netze - zentrale Begriffe wie Recheneinheit und Aktivierungsfunktion kennenlernen
- Lineares NN für Regression: Lineare Regression aka. ordinary least squares

Neuronale Netze (NN)

- Werden **neuronal** genannt, weil ihr Ursprung im McCulloch-Pitts-Neuron (1943) liegt, einem vereinfachten Modell des menschlichen Neurons als eine Art Rechenelement, das mit Aussagenlogik beschrieben werden konnte.



Wikipedia, Egm4313.s12 (Prof. Loc Vu-Quoc) - Own work

Recheneinheit (Unit, Neuron)

- Grundlegender Baustein eines NN ist eine **Recheneinheit**, auch **Unit** genannt. Sie nimmt einen **Vektor reeller Zahlen** als Eingabe, führt eine Berechnung mit ihm durch, und erzeugt eine Ausgabe. Die Berechnung besteht aus zwei Schritten, die hintereinander ausgeführt werden:
- Schritt 1: **gewichtete Summe** der Eingaben, mit einer zusätzlichen additiven Konstante b , sog. **Bias**:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b = b + \sum_{i=1}^n w_i x_i = w \cdot x + b$$

- Schritt 2: **Aktivierung** durch eine nicht-lineare [\[1\]](#) Funktion f , sog. **Aktivierungsfunktion**:

$$y = f(z)$$

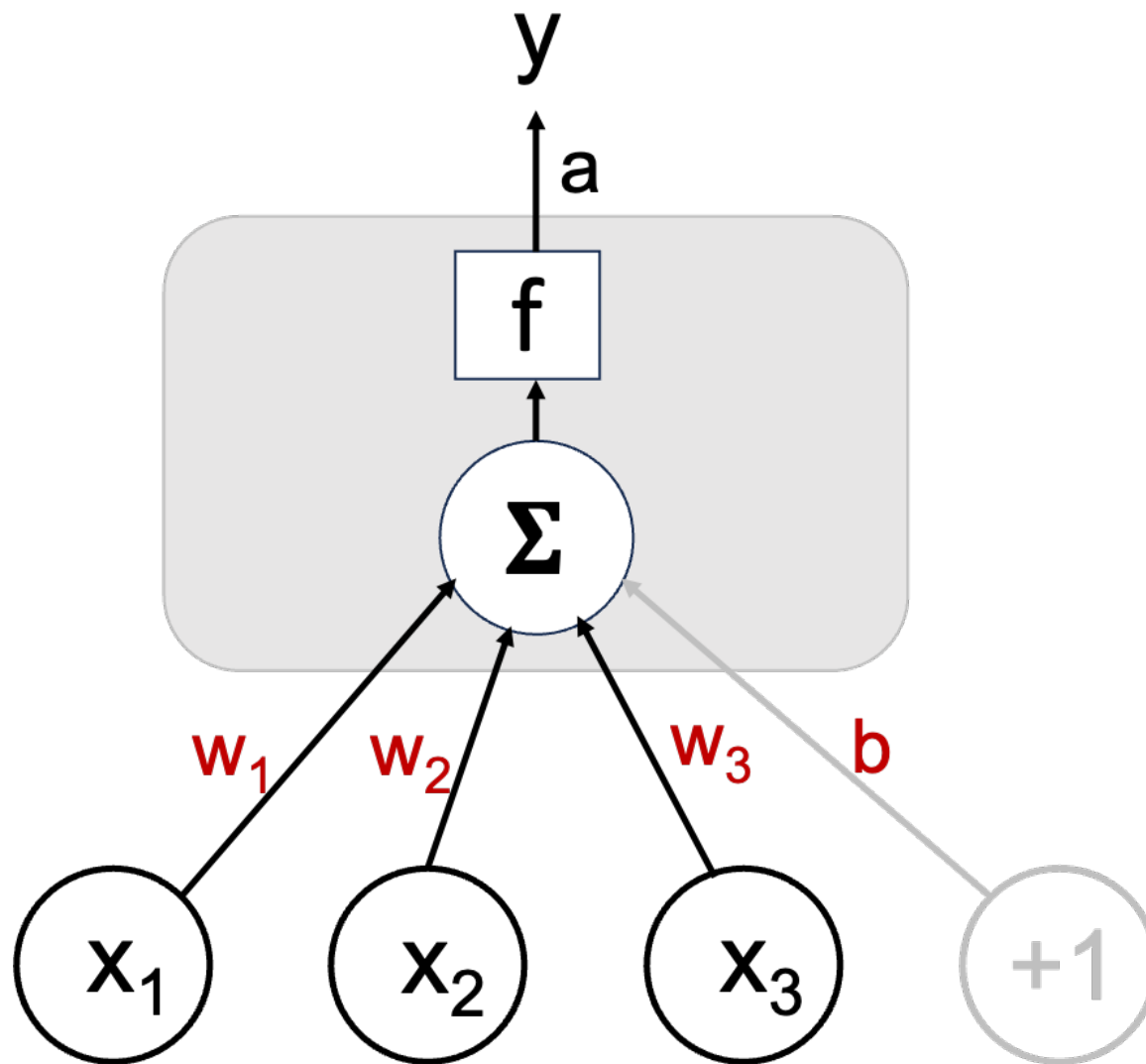
Eine mögliche Aktivierungsfunktion ist die **Sigmoid-Funktion**

$$f(z) = \frac{1}{e^{-z}}$$

Wenn Sigmoid als Aktivierungsfunktion f , dann erhalten wir

$$y = f(z) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

als Berechnung, die in einer Unit stattfindet, wobei x einen Input-Vektor repräsentiert.



Schema einer Unit, mit einem 3-dimensionalen Eingabe-Vektor.

- Unit erhält 3 Input-Werte x_1 , x_2 und x_3 und berechnet eine **gewichtete Summe** mit den jeweiligen Gewichten w_1 , w_2 und w_3

- Der **Bias** b wird dazu addiert; das Zwischenergebnis dieser linearen Operation bezeichnen wir als z
- z wird an die **Aktivierungsfunktion** f (z.B. Sigmoid-Funktion) weitergegeben und das Ergebnis a (für Aktivierung) wird berechnet
- In diesem Fall ist der Output der Unit y gleich dem Output der Aktivierungsfunktion a ; in größeren Netzen bezeichnet y üblicherweise den Output des gesamten Netzes

Aktivierungsfunktion

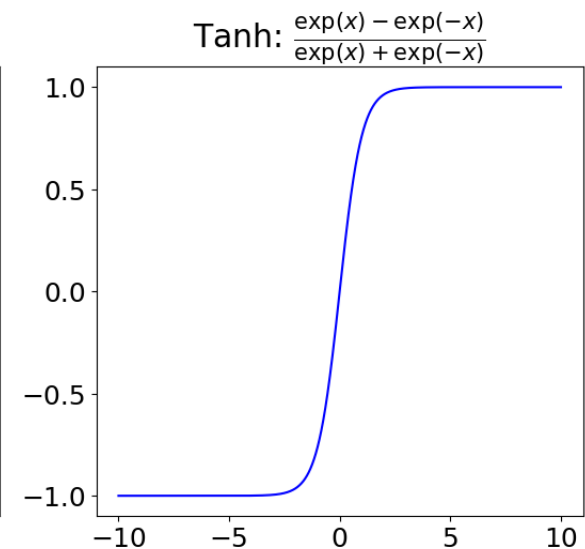
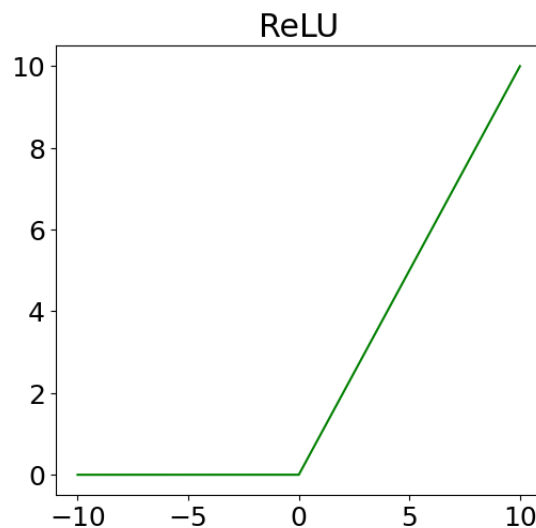
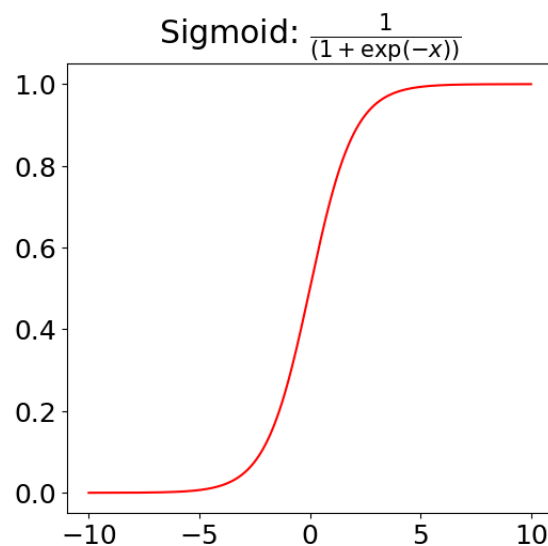
Es gibt drei populäre Aktivierungsfunktionen (aber grundsätzlich ist alles denkbar):

```
from numpy import exp, log, tanh, linspace, sin, float_power
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams.update({'font.size': 18})
```

```
def relu(x):
    return max(0, x)
def sigmoid(x):
    sg = 1/(1+exp(-x))
    return sg
```

```
x = linspace(-10, 10, 10000)
y1 = sigmoid(x)
y2 = [relu(i) for i in x]
y3 = tanh(x)
```

```
plt.figure(1, figsize=(20, 12))
plt.subplot(231)
sns.lineplot(x=x, y=y1, color='red', label='')
plt.title(r'Sigmoid:  $\frac{1}{(1+\exp(-x))}$ ')
plt.subplot(232)
sns.lineplot(x=x, y=y2, color='green', label='')
plt.title('ReLU')
plt.subplot(233)
sns.lineplot(x=x, y=y3, color='blue', label='')
plt.title(r'Tanh:  $\frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$ ')
plt.show()
```



! Übung (3–5 Min.)

Gegeben sei eine Rechen-Unit mit folgenden Parametern

$$w = (0.2, 0.3, 0.9), \quad b = 0.5$$

und Sigmoid als Aktivierungsfunktion. Was wäre der Output für $x = (0.5, 0.6, 0.1)$? Berechnen Sie den Wert der Sigmoid-Funktion nicht direkt, sondern schätzen Sie ihn oder lesen ihn vom Graphen ab.

Der Bias-Term lässt sich folgendermaßen in den Gewichte-Vektor integrieren:

$$w_1 \cdot x_1 + \dots w_n \cdot x_n + b = w_1 \cdot x_1 + \dots w_n \cdot x_n + b \cdot 1 = \tilde{w} \cdot \tilde{x}$$

mit $\tilde{w} = (b, w_0, \dots, w_n)$ und $\tilde{x} = (1, x_0, \dots, x_n)$.

In der schematischen Unit-Darstellung erhält der Input-Layer einen zusätzlichen Knoten für die Konstante 1

Anmerkung zur mathematischen Notation

- Ein Vektor mit n Einträgen kann als eine Matrix der Dimension $1 \times n$ (Zeilenvektor) oder $n \times 1$ (Spaltenvektor) begriffen werden. Beide sind zueinander transponiert.
- Grundsätzlich lassen sich zwei Matrizen $A = (a_{ij})$ und $B = (b_{kl})$ miteinander multiplizieren, wenn ihre Dimensionen kompatibel sind, d.h. die Anzahl der Spalten von A ist gleich der Anzahl der Zeilen von B .
- In den oberen Berechnungen ist der Gewichte-Vektor w ein Zeilenvektor ($1 \times n$ -dim. Matrix) und der Input-Vektor x ein Spaltenvektor ($n \times 1$ -dim. Matrix), d.h. w ist eigentlich w^T . Wenn sich dies aber direkt aus der Berechnung erschließt, sparen wir uns diese Notation bei der Skalarmultiplikation.

Architektur

- Die Units in einem NN werden i.d.R. in **Schichten (Layers)** zusammengefasst
- Verschiedene Schichten können unterschiedliche Transformationen an den Ergebnissen der Units vornehmen
- Input-Werte wandern von der Input-Schicht zur Output-Schicht, und können einzelne Schichten auch mehrfach durchlaufen
- Im einfachsten Fall laufen die Inputs nur in eine Richtung und passieren jede Schicht nur einmal (sog. Feed-Forward NN)
- Unter der **Architektur** eines NN verstehen wir die Gesamtstruktur bzw. das Layout des Netzes, einschließlich der Anzahl und Anordnung der Schichten, der Anzahl der Neuronen in jeder Schicht, der verwendeten Aktivierungsfunktionen und der Verbindungen zwischen den Neuronen
- Es gibt verschiedene Arten von Architekturen, wie z. B. feed forward, recurrent, convolutional, die jeweils ihre eigenen Merkmale und Anwendungen haben. Die Wahl der Architektur hängt von dem spezifischen Problem ab, das gelöst werden soll, sowie von der Art der zu analysierenden Daten

Ein **Perceptron** ist eine sehr einfache Recheneinheit, die binären Output 0 oder 1 produziert:

$$L = \begin{cases} 0 & \text{wenn } w \cdot x + b \leq 0 \\ 1 & \text{wenn } w \cdot x + b > 0 \end{cases}$$

! Übung AND Verknüpfung (3–5 Min.)

Die AND Verknüpfung ist durch folgende Wahrheitstabelle definiert:

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Zeigen Sie, dass AND mit Hilfe eines Perceptrons berechnet werden kann.

✓ Lösung anzeigen

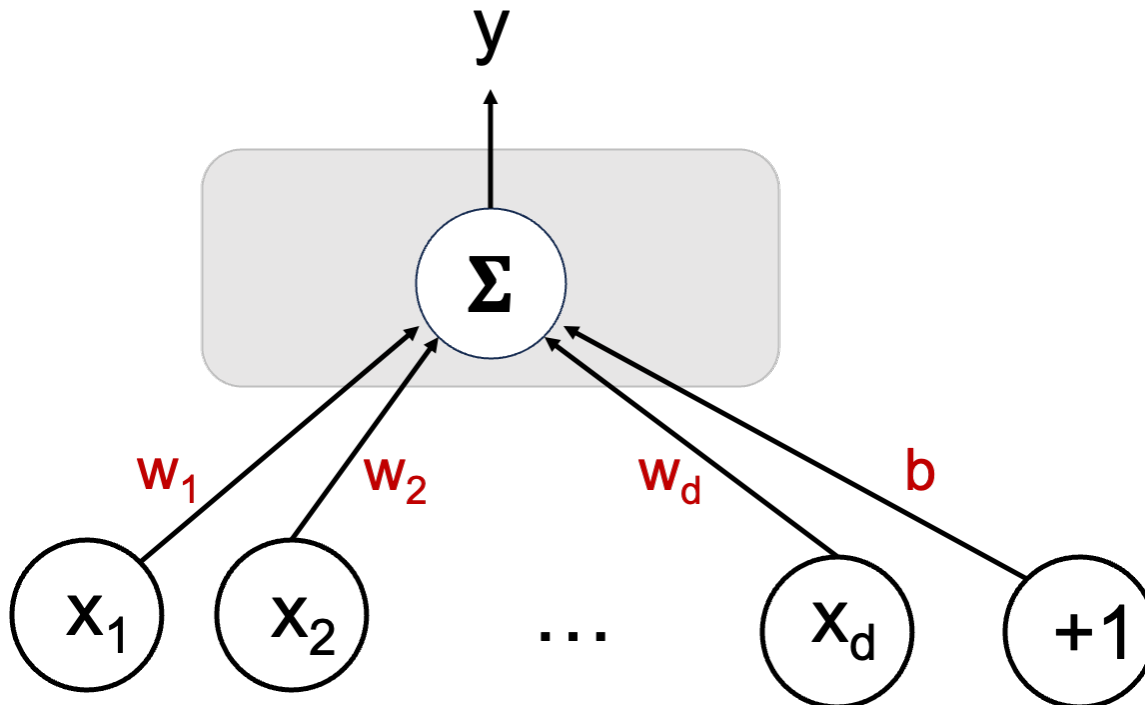
Eine mögliche Wahl ist z. B. $w = (1, 1)$ und $b = -1$:

- $(0, 0): w \cdot x + b = -1 \leq 0 \Rightarrow 0$
- $(0, 1): 0 \leq 0 \Rightarrow 0$
- $(1, 0): 0 \leq 0 \Rightarrow 0$
- $(1, 1): 1 > 0 \Rightarrow 1$

Lineare Neuronale Netze für Regression

Ein flaches, lineares NN kann folgendermaßen für Regression verwendet werden, z.B. für die Vorhersage von Mietpreisen:

- Input: d -dimensionaler Input-Vektor (für Features wie Wohnungsgröße, Alter, Entfernung zum Zentrum, etc.)
- Ein Neuron als Output
- Identität als Aktivierungsfunktion, d.h. $f(z) = z$
- Gewichte $w_1, \dots, w_d$ und Bias b



- Betrachten wir als Beispiel die Vorhersage von Mietpreisen basierend auf Daten eines Immobilienportals

- Aus den Daten extrahieren wir zwei Features: `size` (Größe in qm) und `age` (Alter in Jahren). Das lineare NN resultiert dann in folgendem Hypothesenraum:

$$\text{price} = w_{\text{size}} \cdot \text{size} + w_{\text{age}} \cdot \text{age} + b$$

- Der Bias b bestimmt den geschätzten Preis, wenn alle Features gleich 0 wären (auch wenn das nie passiert). Geometrisch gesprochen, erlaubt der Bias auch Geraden bzw. Hyperflächen zu betrachten, die nicht durch den Ursprung gehen.
- Zur Erinnerung an die letzte Woche: Der Bias lenkt das Modell in eine bestimmte Richtung.
- Unser Ziel ist es, Werte für die Gewichte w und den Bias b zu bestimmen, für die die Modell-Vorhersagen im Durchschnitt möglichst gut zu den echten Werten in unseren Daten passen

Notationen

- Im Allgemeinen notieren wir die Vorhersage $\hat{y}$ als

$$\hat{y} = w_1 \cdot x_1 + \dots w_d \cdot x_d + b \cdot 1 = w^T \cdot x + b,$$

wobei $x \in \mathbb{R}^d$ den Feature-Vektor und $w \in \mathbb{R}^d$ den Gewichte-Vektor eines einzelnen Datenbeispiels bezeichnet.

- Wenn wir einen größeren Datensatz X bestehend aus m Datenvektoren betrachten (z. B. alle Trainingsdaten), können wir folgende Notation verwenden:

$$\hat{y} = Xw,$$

wobei

$$X = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_d^{(m)} & 1 \end{pmatrix} \in \mathbb{R}^{m \times (d+1)}$$

die **Designmatrix** der Eingabedaten ist, deren i -te Zeile Datenvektor $x^{(i)}$ entspricht (hier haben wir b mit in die Gewichtematrix integriert).

Der Vektor der Vorhersagen ergibt sich dann als

$$\hat{y} = \begin{pmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \vdots \\ \hat{y}^{(m)} \end{pmatrix} = Xw \in \mathbb{R}^m.$$

Hier ist also m die Anzahl der Datenbeispiele im Datensatz, $x^{(i)}$ das i -te Beispiel in den Daten, und $x_j^{(i)}$ das j -te Feature des i -ten Beispiels.

Loss Function

Im Training suchen wir Parameter $(w^*) = (w_1^*, \dots, w_d^*, b^*)$, welche den **mittleren quadratischen Fehler (MSE)** auf den Trainingsdaten minimieren, d.h.

$$(w_1^*, \dots, w_d^*, b^*) = \operatorname{argmin}_{w_1, \dots, w_d, b} L(w_1, \dots, w_d, b),$$

wobei

$$\begin{aligned} L(w) &= \frac{1}{m} \cdot \left((\hat{y}^{(1)} - y^{(1)})^2 + \dots + (\hat{y}^{(m)} - y^{(m)})^2 \right) \\ &= \frac{1}{m} \cdot \left((w^T x^{(1)} - y^{(1)})^2 + \dots + (w^T x^{(m)} - y^{(m)})^2 \right) \end{aligned}$$

Training durch Gradientenabstieg

- Die optimalen Parameter $(w_1^*, \dots, w_d^*, b^*)$ findet man durch den sog. **Gradientenabstieg (gradient descent)** im Fehlerraum -Die Idee ist, mit zufälligen Parametern zu starten und diese iterativ so anzupassen, dass dadurch der Wert der Fehlerfunktion auf den Trainingsdaten möglichst schnell reduziert wird





Idee des Gradientenabstiegs

- Wir betrachten die Fehlerlandschaft als Berglandschaft. Aber wir sehen den Berg nicht von oben sondern nur ein winziges Stück direkt um uns herum.
- In diesem kleinen Ausschnitt ist die Landschaft fast linear (Tangentialebene).
- Bei einer linearen Funktion zeigt der sog. **Gradient** ∇ genau entlang der Linie, auf der die Funktion am stärksten steigt - wie die Tangente bei Funktionen mit einer Unbekannten. Wir gehen deshalb in die entgegengesetzte Richtung, denn wir wollen auf dem steilsten Weg ins Tal.
- Beim Optimieren in NN kennen wir auch nicht die globale Fehler-Landschaft; der Gradient Descent nutzt deshalb nur lokale Informationen
- Die Schrittweite, mit welcher wir in Richtung des negativen Gradienten laufen, ist wichtig: wenn wir nur einen kleinen Schritt laufen, bis wir wieder den Gradienten ermitteln, müssen wir viele Berechnungen machen; wenn wir zu weit gehen, verpassen wir evtl. das Tal. Die Schrittweite heißt auch **Lernrate, learning rate**).
- Der Algorithmus sieht also in etwa so aus:

$$(w) \leftarrow (w) - \eta \cdot \nabla L(w),$$

wobei $\eta > 0$ die Lernrate bezeichnet.

- Der Gradient ist eine **Verallgemeinerung der Steigung für Funktionen in mehr-dimensionalen Räumen**
- Für 1-dim Funktionen entspricht er der **1. Ableitung**; in n -dim. Räumen berechnen wir stattdessen **partielle Ableitungen**. Hierfür betrachten Sie jeweils eines der Features als Variable und die restlichen als Konstanten.
- Der Gradientenabstieg spielt für das Lösen der Minimierungsaufgabe bei nahezu jedem NN eine fundamentale Rolle

! Übung (3–5 Min.)

Angenommen, folgende Fehlerfunktion ergibt sich, nachdem alle Trainingsdaten durch die initiale Version des Neuronalen Netzes durchlaufen sind:

$$L(w) = L(w_1, w_2) = w_1 + 2w_2^2 + 4$$

Berechnen Sie die partiellen Ableitungen von L .

Lineare Regression

- Das vorgestellte Verfahren – lineares neuronales Netz mit Identitätsaktivierung, einem einzelnen Output-Neuron und MSE als Loss – entspricht der **Linearen Regression**, auch **Ordinary Least Squares (OLS)** genannt.
- Eines der populärsten und einfachsten Verfahren für Regressionsaufgaben (seit dem 19. Jahrhundert).

Training der Linearen Regression

Lineare Regression kann auf zwei Arten trainiert werden:

Gradient Descent (GD)

- Optimiert die MSE-Loss-Funktion iterativ, wie oben gezeigt:

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

- Funktioniert auch für große Datensätze.

- Grundlage für *alle* neuronalen Netze.

Geschlossene Lösung (Normalengleichung)

$$w^* = (X^\top X)^{-1} X^\top y$$

- Direkte, analytische Lösung des Minimierungsproblems.
- Einfach zu implementieren.
- Wird ineffizient oder numerisch instabil, wenn:
 - viele Features vorhanden sind,
 - die Matrix $X^\top X$ fast singulär ist.

Interpretation der Gewichte

- Jedes Gewicht w_j beschreibt:

„Wie stark ändert sich die Vorhersage im Mittel, wenn Feature x_j steigt?“

- Vorzeichen:
 - $w_j > 0$: höheres Feature $\rightarrow$ höhere Vorhersage
 - $w_j < 0$: höheres Feature $\rightarrow$ niedrigere Vorhersage
- Betrag:
 - Je größer $|w_j|$, desto stärker der Einfluss des Features.

Lineare Regression gilt daher als **gut interpretierbar**.

Annahmen der Linearen Regression

Statistiker*innen formulieren üblicherweise vier zentrale Annahmen. Diese sind wichtig für **statistische Interpretation**, weniger für **Machine Learning**. Der Vollständigkeit halber sind sie hier aufgeführt:

1. Linearität

Der bedingte Erwartungswert soll linear sein:

$$E[Y \mid X = x] = w^T x + b$$

Das bedeutet nicht, dass die Daten perfekt auf einer Geraden liegen müssen – nur, dass der *systematische Zusammenhang* näherungsweise linear ist.

2. Unabhängiges, identisch verteiltes Rauschen (i.i.d.)

Die Fehlerterme sind unabhängig voneinander und haben dieselbe Verteilung.

3. Homoskedastizität

Die Varianz des Rauschens ist konstant:

$$\text{Var}(\varepsilon \mid X = x) = \sigma^2$$

4. Normalverteiltes Rauschen

Die Fehlerterme sind (für statistische Tests) normalverteilt.

Für reine Vorhersage ist diese Annahme *nicht notwendig*.

Zu 3.: Beispiel für Heteroskedastizität

- Heteroskedastizität bedeutet, dass die **Streuung der Fehler nicht konstant** ist.
Die Varianz des Fehlers hängt also vom Wert der Features ab.
- Ein anschauliches Beispiel ist Vorhersage des Gehalts anhand der Berufserfahrung
- Nehmen wir hierfür an, wir wollen das jährliche Gehalt Y anhand der Berufserfahrung (in Jahren) X vorhersagen.
Ein typischer Befund aus realen Daten wäre:
 - Berufseinsteiger (0–5 Jahre) verdienen alle ähnliche Gehälter → geringe Streuung

- Senior-Fachkräfte (10–20 Jahre) verdienen sehr unterschiedlich → höhere Streuung
- Grafisch sähe das so aus: die Punkte liegen **links sehr eng** und **rechts breit gestreut** – wie ein Trichter.

Was passiert, wenn die Annahmen nicht gelten?

1. Linearitätsannahme verletzt

- Die Beziehung zwischen Features und Zielvariable ist nicht linear
- Folge: Das Modell ist systematisch verzerrt (**Bias**)
- Vorhersagen sind oft dennoch brauchbar, aber suboptimal
- Lösungsmöglichkeiten:
 - Feature-Engineering (Polynome, Interaktionsterme)
 - Nichtlineare Modelle

2. Nicht-i.i.d. Daten

Typisches Beispiel: Zeitreihen oder räumlich abhängige Daten.

- Folge: Die Unsicherheitsabschätzungen sind falsch.
- Vorhersage kann trotzdem funktionieren, aber man muss vorsichtig sein.
- Lösungen:
 - Zeitreihenmodelle
 - Cross-Validation mit zeitlicher Struktur

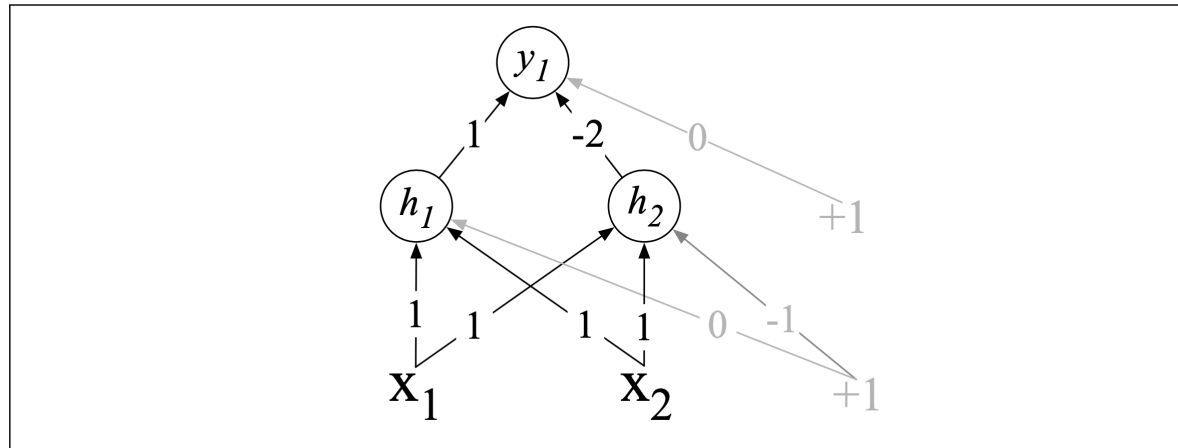
Aufgaben

Aufgabe 1 - OR als Perzeptron

Stellen Sie die Wahrheitstabelle für die OR-Verknüpfung auf und zeigen Sie, dass OR als Perzeptron dargestellt werden kann.

Aufgabe 2 - XOR als mehrschichtiges Neuronales Netz

Schon früh wurde erkannt, dass die Leistung neuronaler Netze, wie bei den echten Neuronen, aus der Kombination der Rechen-Einheiten zu größeren Netzwerken kommt. Eine der geschicktesten Demonstrationen der Notwendigkeit von mehrschichtigen Netzwerken war der Beweis von Minsky und Papert (1969), dass eine einzelne neuronale Einheit einige sehr einfache Funktionen wie XOR aus dem Input nicht berechnen kann. Prüfen Sie nach, dass folgendes Netzwerk mit einem hidden Layer und ReLU als Aktivierungsfunktion für die Inputs $(0, 0)$ und $(0, 1)$ den korrekten Output generiert.



Aufgabe 3 - Lineare regression implementieren

Trainieren Sie eine Lineare Regression anhand des Palmer Penguins Datensatzes. Sie können die Features sowie die Zielvariable selbst bestimmen. Nutzen Sie hierfür die scikit-learn Klasse `sklearn.linear_model.LinearRegression`. Geben Sie sich anschließend die Gewichte und den Bias des trainierten Modells und interpretieren Sie das Ergebnis.

-
- [1] Lineare Aktivierungsfunktionen machen nicht viel Sinn, da die Hintereinanderausführung linearer Funktionen wieder eine lineare Funktion ist.